

What is Alpine JS ?

Alpine is a rugged, minimal tool for composing behavior directly in your markup. Think of it like jQuery for the modern web. Plop in a script tag and get going.

Installation:-

CDN -

```
<script defer  
src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js"></script>
```

```
<script defer  
src="https://cdn.jsdelivr.net/npm/alpinejs@3.14.8/dist/cdn.min.js"></script>
```

NPM -

```
npm install alpinejs
```

State:-

State (JavaScript data that Alpine watches for changes) is at the core of everything you do in Alpine. You can provide local data to a chunk of HTML, or make it globally available for use anywhere on a page using `x-data` or `Alpine.store()` respectively.

x-data:-

Everything in Alpine starts with the `x-data` directive.

`x-data` defines a chunk of HTML as an Alpine component and provides the reactive data for that component to reference.

Properties defined in an `x-data` directive are available to all element children. Even ones inside other, nested `x-data` components.

x-text:-

`x-text` sets the text content of an element to the result of a given expression.

x-on:-

x-on allows you to easily run code on dispatched DOM events. Its shorthand is @

x-show:-

x-show provides an expressive way to show and hide DOM elements.

If you want to apply smooth transitions to the x-show behavior, you can use it in conjunction with x-transition.

x-cloak:-

Sometimes, when you're using AlpineJS for a part of your template, there is a "blip" where you might see your uninitialized template after the page loads, but before Alpine loads.

x-cloak addresses this scenario by hiding the element it's attached to until Alpine is fully loaded on the page.

x-if:-

x-if is used for toggling elements on the page, similarly to x-show, however it completely adds and removes the element it's applied to rather than just changing its CSS display property to "none".

Unlike x-show, x-if, does NOT support transitioning toggles with x-transition.

x-if should not be applied directly to the element, but instead to a <template> tag that encloses the element.

<template> tags can only contain one root element.

x-init:-

The x-init directive allows you to hook into the initialization phase of any element in Alpine.

x-bind:-

x-bind allows you to set HTML attributes on elements based on the result of JavaScript expressions.

x-html:-

x-html sets the "innerHTML" property of an element to the result of a given expression.

x-for:-

Alpine's x-for directive allows you to create DOM elements by iterating through a list.

It is important to specify unique keys for each x-for iteration if you are going to be re-ordering items.

x-effect:-

x-effect is a useful directive for re-evaluating an expression when one of its dependencies change.

It watches for changes and runs effects.

x-ignore:-

If for some reason, you don't want Alpine to touch a specific section of your HTML, you can prevent it from doing so using x-ignore.

x-ref:-

x-ref in combination with \$refs is a useful utility for easily accessing DOM elements directly. It's most useful as a replacement for APIs like getElementById and querySelector.

x-teleport:-

The x-teleport directive allows you to transport part of your Alpine template to another part of the DOM on the page entirely.

x-id:-

It allows you to declare a new "scope" for any new IDs generated using \$id(). It accepts an array of strings (ID names) and adds a suffix to each \$id('...') generated within it that is unique to other IDs on the page.

x-model:-

x-model allows you to bind the value of an input element to Alpine data.

x-model is two-way bound, meaning it both "sets" and "gets".

x-model works with the following input elements:

`<input type="text">`

`<textarea>`

`<input type="checkbox">`

`<input type="radio">`

`<select>`

`<input type="range">`

\$el:-

It is a magic property that can be used to retrieve the current DOM node.

\$refs:-

It is a magic property that can be used to retrieve DOM elements marked with x-ref inside the component. This is useful when you need to manually manipulate DOM elements. It's often used as a more succinct, scoped, alternative to document.querySelector.

\$data:-

\$data is a magic property that gives you access to the current Alpine data scope (generally provided by x-data).

\$root:-

\$root is a magic property that can be used to retrieve the root element of any Alpine component. In other words the closest element up the DOM tree that contains x-data.

\$watch:-

You can "watch" a component property using the \$watch magic method.

\$dispatch:-

\$dispatch is a helpful shortcut for dispatching browser events.

\$store:-

You can use \$store to conveniently access global Alpine stores registered using Alpine.store(...)

Alpine.data:-

Alpine.data(...) provides a way to re-use x-data contexts within your application.

Alpine.bind:-

Alpine.bind(...) provides a way to re-use x-bind objects within your application.